

Plan de Pruebas — SIGAI-SES v1.0.0

1. Objetivo

Garantizar la calidad, estabilidad y seguridad del Sistema Integral de Gestion de Activos e Inventario (SIGAI-SES) mediante la ejecucion de pruebas unitarias, de integracion, funcionales y de rendimiento.

2. Alcance

Este plan cubre todos los modulos del sistema:

Modulo	Cobertura
Autenticacion y Autorizacion	Login, JWT, roles, permisos, cierre de sesion
Gestion de Usuarios	CRUD, asignacion de roles, activacion/desactivacion
Catalogo de Items	CRUD, validacion de referencias unicas, soft delete
Activos	CRUD, validacion de serial unico, cambios de estado
Inventario Bulk	Control de stock, alertas de minimo, movimientos
Clientes	CRUD, validacion de NIT, soft delete
Proyectos	CRUD, asociacion con clientes, cambios de estado
Proveedores	CRUD, categorias, soft delete
Garantias	Flujo completo de estados, calculo de alertas de estancamiento
Actas de Entrega	Creacion con detalles, generacion de PDF, firmas
Desmontes y Triaje	Registro masivo, calificacion tecnica, cambio de estado
Movimientos (Kardex)	Registro automatico, integridad transaccional
Alertas	Reglas, generacion automatica, cambios de estado
Auditoria	Registro de todas las acciones CRUD
Importacion Excel	Carga masiva con upsert, deteccion de tipo, validacion
Reportes	Exportacion Excel y PDF, streaming de grandes volumenes
Busqueda Global	Resultados por tipo de entidad, minimo 2 caracteres

3. Tipos de Pruebas

3.1 Pruebas Unitarias

Herramienta: pytest + pytest-asyncio **Cobertura objetivo:** >= 80%

Componente	Casos a probar
<code>core/security.py</code>	Creacion y validacion de JWT, hash de contraseñas, refresh tokens
<code>crud/*.py</code>	Operaciones CRUD basicas, validaciones de unicidad, soft delete
<code>services/import_service.py</code>	Parseo de Excel, upsert, normalizacion, deteccion de tipo
<code>api/deps.py</code>	Validacion de tokens, obtencion de usuario actual
<code>schemas/*.py</code>	Validacion de campos obligatorios, formatos, enumeraciones

3.2 Pruebas de Integracion

Herramienta: pytest + httpx (TestClient asincrono) **Base de datos:** MySQL de prueba o SQLite

Escenario	Descripcion
Flujo completo de autenticacion	Login -> Obtener perfil -> Refresh token -> Logout
CRUD de inventario	Crear item -> Crear activo -> Consultar -> Actualizar -> Eliminar
Importacion Excel	Cargar archivo -> Verificar upsert -> Verificar resumen
Flujo de garantia	Crear caso -> Enviar a proveedor -> Recibir -> Resolver -> Entregar
Generacion de acta	Crear acta con detalles -> Generar PDF -> Descargar
Alerta automatica	Configurar regla -> Reducir stock bajo minimo -> Verificar alerta generada

3.3 Pruebas Funcionales (E2E)

Herramienta: Manual + scripts de automatizacion (Selenium/Playwright - pendiente)

Modulo	Casos de Prueba
Login	Credenciales validas, invalidas, usuario inactivo, token expirado
Dashboard	Visualizacion de metricas, busqueda global
Inventario	Filtros, paginacion, creacion, edicion, eliminacion, importacion
Garantias	Creacion, cambio de estados, alerta de estancamiento
Actas de Entrega	Creacion con detalles, captura de firma, generacion PDF
Desmontes	Registro masivo, triaje, cambio de estado
Alertas	Visualizacion, filtros, cambio de estado, resolucion
Usuarios	CRUD, cambio de rol, activacion/desactivacion
Reportes	Exportacion Excel, exportacion PDF, grandes volúmenes

3.4 Pruebas de Rendimiento

Herramienta: Locust (archivo `locustfile.py` existente)

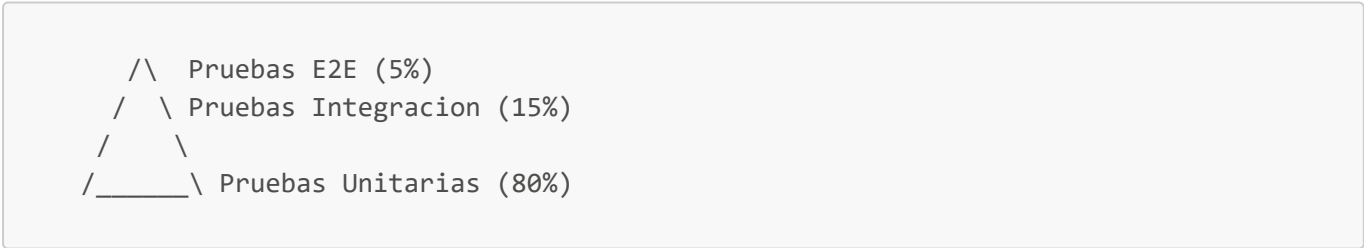
Escenario	Metrica Objetivo
100 usuarios concurrentes consultando inventario	Respuesta < 500ms
50 usuarios concurrentes importando Excel	Procesamiento < 30s
Exportacion de 50,000 registros a Excel	Generacion < 10s
Pico de 200 peticiones simultaneas	Sin errores 5xx, CPU < 80%

3.5 Pruebas de Seguridad

Prueba	Descripcion
Inyeccion SQL	Validar que parametros estan sanitizados por SQLAlchemy
XSS	Validar que las respuestas JSON no ejecutan scripts
JWT tampering	Verificar que tokens modificados son rechazados
Acceso no autorizado	Verificar que endpoints protegidos requieren token valido
Fuerza bruta	Verificar rate limiting (pendiente de implementar)
OWASP Top 10	Evaluacion de cumplimiento (programada trimestralmente)

4. Estrategia de Pruebas

4.1 Piramide de Pruebas



4.2 Flujo de Pruebas en CI/CD (GitHub Actions)



```
Analisis Estatico (mypy / bandit)
|
v
Build Frontend (npm run build)
|
v
Pruebas de Humo en Staging
|
v
Despliegue a Produccion
```

5. Criterios de Aceptacion

5.1 Criterios de Entrada

- Código compilado sin errores
- Pruebas unitarias ejecutadas localmente
- Code review aprobado (Pull Request)
- Documentación actualizada

5.2 Criterios de Salida

- Cobertura de pruebas $\geq 80\%$
- 0 errores críticos o de alta prioridad
- Tiempo de respuesta $< 200\text{ms}$ en operaciones críticas
- Tasa de fallos $< 0.5\%$
- Pruebas de seguridad sin hallazgos críticos

6. Herramientas

Herramienta	Propósito
pytest 9.0+	Framework de pruebas unitarias
pytest-asyncio	Soporte para código asíncrono
pytest-cov	Medición de cobertura
httpx	Cliente HTTP para pruebas de integración
Locust	Pruebas de rendimiento
GitHub Actions	Integración continua
ruff	Linter de Python
ESLint	Linter de TypeScript/React
bandit	Análisis de seguridad estático
coverage.py	Reporte de cobertura

7. Cronograma

Fase	Duracion	Actividades
Pruebas Unitarias v1	Semana 1-2	Escribir tests para modelos, CRUD y schemas
Pruebas Integracion v1	Semana 3-4	Probar flujos completos de API
Pruebas Funcionales	Semana 5-6	Validar modulos desde interfaz de usuario
Pruebas Rendimiento	Semana 7	Ejecutar Locust, optimizar cuellos de botella
Pruebas Seguridad	Semana 8	Escaneo OWASP, pruebas de penetracion
Ajustes y Cierre	Semana 9-10	Correccion de defectos, reporte final

Documento actualizado al: Julio 2026 v1.0